

```

#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {
    if (argc != 2) return 1;

    /* * fopen(filename, mode)
     * Öffnet eine Datei.
     * Mode "rb" = read binary (verhindert Probleme mit Windows/Linux Zeilenumbrüchen).
     * Return: File-Pointer (FILE *) oder NULL bei Fehler.
     */
    FILE *fd = fopen(argv[1], "rb");
    if (fd == NULL) return 1;

    /* fseek(pointer, offset, position)
     * Verschiebt den internen Lesezeiger. SEEK_END springt ans Ende. */
    fseek(fd, 0, SEEK_END);

    /* ftell(pointer)
     * Gibt die aktuelle Position des Zeigers in Bytes zurück (hier = Dateigröße).
     */
    long file_size = ftell(fd);

    /* rewind(pointer)
     * Spult den Zeiger zurück an den Anfang (SEEK_SET, 0). Wichtig für das anschließende Lesen! */
    rewind(fd);

    /* malloc(size)
     * Reserviert einen Block Speicher. size in Bytes.
     * +1 für das Null-Terminierungs-Byte ('\0') bei Strings! */
    char *buffer = malloc(file_size + 1);
    if (buffer == NULL) {
        fclose(fd);
        return -1;
    }

    /* fread(destination, size_of_element, number_of_elements, file_pointer)
     * Liest Datenblock aus der Datei in den Speicher.
     * Return: Anzahl der erfolgreich gelesenen Elemente. */
    size_t read_bytes = fread(buffer, 1, file_size, fd);
    buffer[read_bytes] = '\0'; // String sauber abschließen

    fclose(fd); // Datei schließen, sobald alles im RAM ist

    // ... hier passiert die Magie ...

    /* free(pointer)
     * Gibt den mit malloc reservierten Speicher wieder an das OS zurück. */
    free(buffer);
    return 0;
}

```